

Ballistics Mapper

Ansh Menghani

September 2026

Abstract

This project develops a physics-based system for predicting and controlling projectile trajectories from a moving platform. A physics model incorporating gravity, aerodynamic drag, the Magnus effect, and rotational drag is first used to describe the projectile's motion. The ordinary differential equations of the projectile's motion are derived from Newton's second law and are numerically approximated using a fourth-order Runge–Kutta method. A Levenberg–Marquardt inverse solver determines the launch conditions required to reach a specified target. These solutions are precomputed across a range of target positions and initial velocities of the platform (for shoot-on-the-move situations) to generate trajectory surfaces. A valid-shot selection process then identifies feasible combinations of launch conditions and operating parameters. Finally, the precomputed solutions are approximated using fourth-degree polynomial models, providing a computationally efficient method for determining launch angles during operation. The resulting framework combines detailed physics-based simulation with numerical optimization and compact polynomial approximations, enabling real-time projectile trajectory prediction while accounting for motion of the launching platform.

1 The Game

1.1 FRC 2026: Rebuilt

The FIRST Robotics Competition (FRC) is a high-school-level robotics competition where teams build robots and compete against each other in a 3v3 matchup where robots aim to complete tasks with the highest efficiency. One of the main tasks in the 2026 FRC game was to use your robot to launch a spherical high-density foam projectile, called “Fuel”, into a basketball-like hoop, called “The Hub”. In this paper, I will simply use the terms ball and hoop for Fuel and The Hub, respectively.

1.2 Introduction

Teams had to determine the exit velocity and direction that the robot needed to impart on the ball to sink the shot. This required accounting for the robot's distance, orientation, and velocity relative to the hoop. Most teams designed a turret or drivetrain algorithm to align their shooter with the hoop whenever shooting. If the robot is stationary, its orientation can be used to directly align the shooter with the hoop, eliminating the need to compensate for the robot's motion. However, when shooting while moving, the robot's velocity must also be considered. Its velocity can be decomposed into radial and tangential components relative to the hoop. The radial component changes the ball's effective launch velocity toward or away from the hoop and therefore affects the required elevation angle of the shooter, while the tangential component changes the direction in which the ball must be launched and therefore determines the required azimuthal turret angle. Thus, a moving shot requires calculating the robot's position and velocity relative to the hoop, using the distance and radial velocity to determine the shooter's elevation angle and the tangential velocity to determine the azimuthal angle. The turret can then independently command these two angles, allowing the robot to shoot accurately without first stopping or rotating to face the hoop.

1.3 The Challenge

Given the robot's distance, radial velocity, and tangential velocity relative to the hoop, we must determine the shooter speed, elevation angle, and azimuthal angle required to sink the shot.

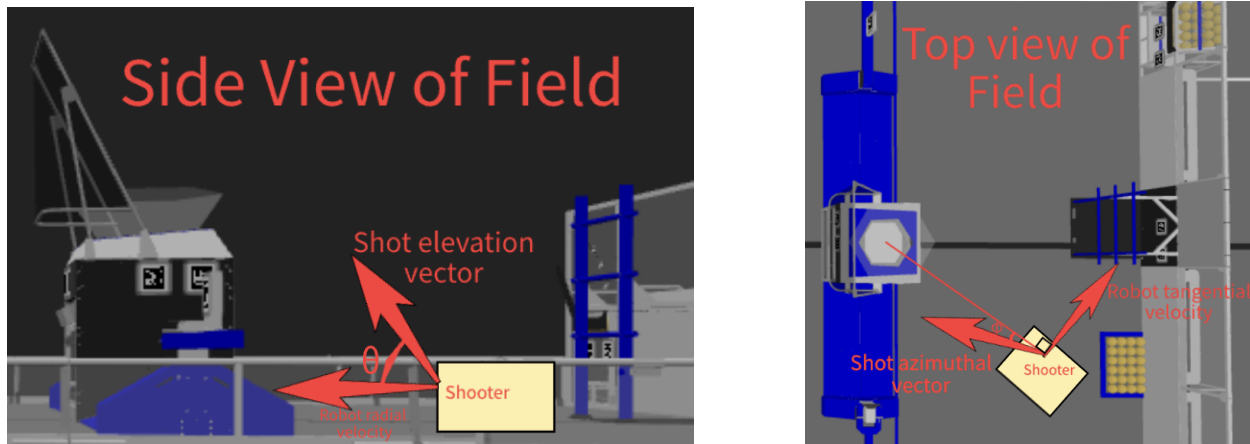


Figure 1: Tangential and radial components of robot velocity and shot.

1.4 Solutions

Most teams choose to empirically determine the right shooter speeds for their robot at different distances and robot velocities relative to the hoop. They find various pairs of distances, velocities, launch speeds, and launch angles that work, then fit an empirical model to use while in-game. This works well. However, in 2026, Team 900 The Zebracorns used a physics model derived from Newton’s second law to predict the path of a ball given its initial xyz velocities and top/backspin (since that is the only spin the shooter can physically impart on the ball). Using this model, we can predict the landing location of a ball given its initial motion, but how can we predict the desired initial shot speed and spin given the target landing location?

We solve this as an inverse problem. Rather than collecting a large set of successful shots and training a regression model to approximate the answer, we take our physics model and work backward from the desired landing location. We use a nonlinear least-squares solver, specifically the Levenberg–Marquardt algorithm, to iteratively guess the ball’s initial conditions, simulate the resulting trajectory with our physics model, and compare the predicted landing location to the target. The solver then adjusts the initial shot speed and spin to minimize this error, repeating until the simulated trajectory reaches the desired target. In other words, instead of learning a mapping from data, we use the physics model itself to numerically solve for the inputs that produce the desired output.

2 Ballistics Mapper

2.1 Projectile Motion Parametric ODEs

We model the ball’s flight by applying Newton’s second law independently in the x , y , and z directions and accounting for the aerodynamic forces acting on the ball. In addition to gravity, the model includes aerodynamic drag and the Magnus force resulting from the ball’s topspin or backspin. The ball is assumed to have rotational velocity only about the z -axis because the robot’s shooter can physically impart spin only in the topspin or backspin direction; it cannot independently impart spin about the other two axes. We also model the aerodynamic drag torque acting on the ball, which causes its spin rate to decay over time. These forces and torques are used to formulate a system of parametric ordinary differential equations (ODEs) describing the ball’s translational velocity and rotational velocity throughout its flight. The robot’s motion relative to the Hub is represented using radial and tangential velocity components. These components are converted in the code into Cartesian x , y , and z velocity components before being passed to the projectile model, with all linear velocities expressed in meters per second.

Forces and torques accounted for:

1. Gravity
2. Drag
3. Magnus Force (lift)
4. Drag Torque

Assumptions:

- Gravity acts along the $-y$ direction.
- Only spin about the z -axis is present (i.e., topspin or backspin).

$$\begin{aligned}
\frac{dv_x}{dt} &= \frac{-\rho A |\vec{v}|}{2m} [C_d v_x + C_L v_y] \\
\frac{dv_y}{dt} &= \frac{\rho A |\vec{v}|}{2m} [-C_d + C_L v_x] - g \\
\frac{dv_z}{dt} &= -\frac{\rho}{2m} C_d A |\vec{v}| v_z \\
\vec{\omega}_z(t) &= \frac{\hat{n}}{\frac{1}{\omega_0} + \frac{t}{2I} C_\omega \rho R^5} \\
S(t) &= \frac{|\vec{\omega}_z(t)| R}{|\vec{v}(t)|} \\
Re(t) &= \frac{\rho |\vec{v}(t)| 2R}{\mu}
\end{aligned}$$

$\vec{v}$ Ball velocity vector, with magnitude $|\vec{v}|$.

$\vec{v}_x$ Ball velocity component in the x direction.

$\vec{v}_y$ Ball velocity component in the y direction.

$\vec{v}_z$ Ball velocity component in the z direction.

$\vec{\omega}_z(t)$ Ball angular velocity about the z -axis as a function of time.

ω_0 Initial angular velocity of the ball.

$\hat{n}$ Unit vector defining the direction of rotation about the z -axis; its sign distinguishes topspin from backspin.

t Time since the ball was launched.

m Mass of the ball.

A Cross-sectional area of the ball.

R Radius of the ball.

I Moment of inertia of the ball about the z -axis.

ρ Density of air.

μ Dynamic viscosity of air.

C_d Coefficient of aerodynamic drag.

C_L Coefficient of lift (Magnus force).

C_ω Coefficient of aerodynamic drag torque.

g Gravitational acceleration.

$S(t)$ Spin ratio of the ball as a function of time, defined as the ratio of the ball's surface velocity due to spin to its translational velocity.

$Re(t)$ Reynolds number of the ball as a function of time, characterizing the flow regime around the ball.

2.2 Numerical Projectile Simulation

The forward model determines the trajectory of the ball from a given set of initial conditions. The nonlinear ordinary differential equations derived from the forces and torques acting on the ball describe the evolution of its velocity in the x , y , and z directions. Because these equations depend on the ball's instantaneous velocity, Reynolds number, spin rate, and spin ratio, they do not have a convenient closed-form solution. We therefore numerically integrate the equations using a fourth-order Runge–Kutta (RK4) method.

For a given initial velocity vector (v_x, v_y, v_z) and initial angular velocity ω , the simulation first integrates the velocity equations forward in time. At each timestep, the aerodynamic coefficients and forces are evaluated using the ball's current state. The resulting velocity history is then integrated a second time to obtain the ball's position as a function of time. The simulation can be stopped when the ball reaches the target height while descending, allowing the resulting x , y , and z coordinates to be treated as the predicted landing location.

The RK4 method evaluates the differential equation four times within each timestep and uses a weighted average of these evaluations to estimate the state at the next timestep. This provides a substantially more accurate approximation than a simple Euler integration while remaining computationally efficient enough to repeatedly evaluate during the inverse-solving process. Thus, the RK4 simulation serves as the forward model: given the launch conditions, it predicts the resulting trajectory and landing location.

Below is an example of the v_x , v_y , v_z , x , y , z , and x, y, z, t plots that the RK4 simulation generates. These plots were generated with the following initial conditions of the ball and the simulator (a positive ω indicates backspin while a negative ω indicates topspin):

radius (m)	mass (kg)	v_{x0} (m/s)	v_{y0} (m/s)	v_{z0} (m/s)	ω_0 (rad/s)	dt
0.1	0.3	8	9	−4	90	0.01

Table 1: Initial conditions of the simulation in Figures 2, 3, and 4.

There are many more parameters that can be set in the original code (initial position, frames of reference, bounds, stopping conditions etc.) For more information, check the project's repository (lined in Section 4).

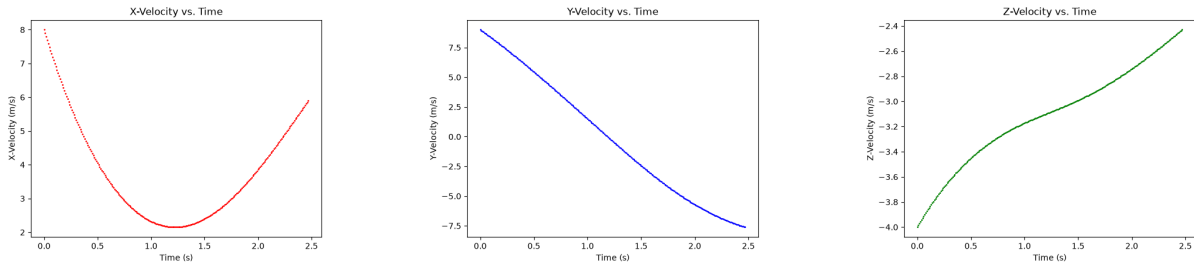


Figure 2: Velocities of ball until $y = 0$.

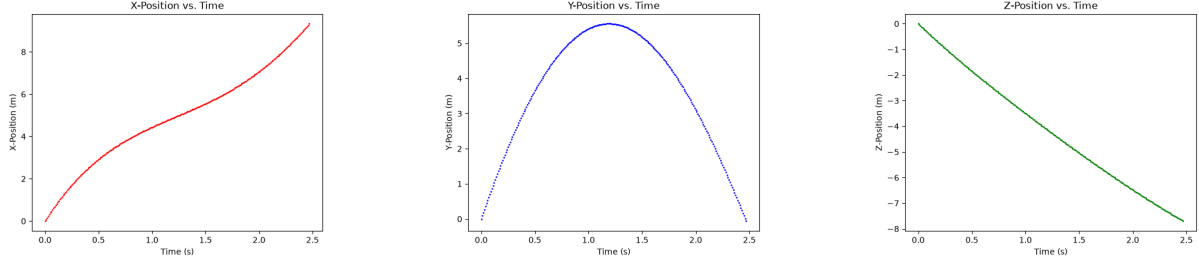


Figure 3: Positions of ball until $y = 0$.

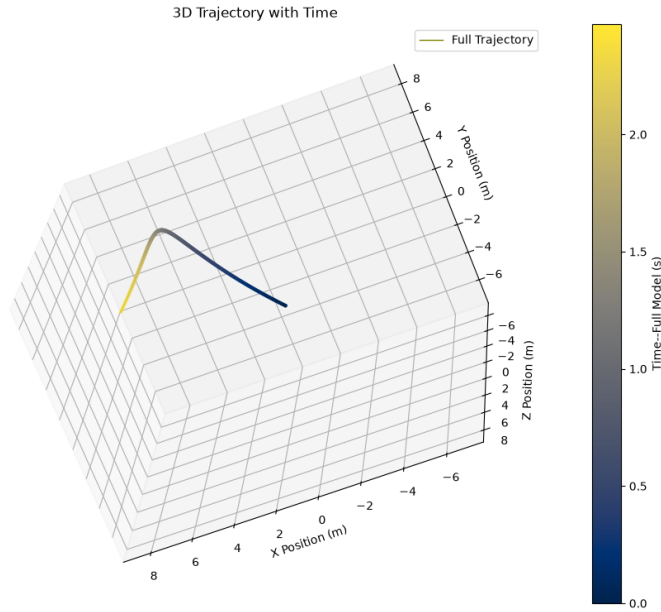


Figure 4: 3D representation of ball path from $y = 0$ to $y = 0$ with time color coded. View is tilted to show the effects of non-gravitational forces on the trajectory.

2.3 Inverse Ballistics Using Levenberg–Marquardt

While the forward simulation determines the landing location from a given set of launch conditions, our objective is the reverse: given a desired target location, determine the launch conditions required for the ball to reach that target. Specifically, the solver can vary the initial velocity magnitude, launch angles θ and ϕ , and initial angular velocity ω to find a shot that reaches the specified x , y , and z coordinates. In this paper, θ denotes the elevation angle and ϕ denotes the azimuthal angle. This is an inverse problem because the desired output of the forward model is known, while the inputs that produce that output are unknown.

A closed-form inverse is impractical because the forward model itself is a numerical integration of nonlinear differential equations. Instead, we solve the inverse problem iteratively using the Levenberg–Marquardt (LM) algorithm. The solver begins with an initial guess for the launch conditions, converts the velocity magnitude and angles into the corresponding x , y , and z velocity components, and passes these initial conditions into

the RK4 simulation. The resulting landing location is compared with the desired target location to produce a residual vector.

The solver then estimates how sensitive each component of this residual is to each adjustable launch parameter. This is accomplished numerically by perturbing each active parameter by a small amount and rerunning the forward simulation. The resulting changes in the residual form a numerical Jacobian, which describes how changes in launch speed, angle, or spin affect the predicted landing location.

Levenberg–Marquardt uses this Jacobian to calculate a parameter update that reduces the squared magnitude of the residual. The algorithm combines the fast convergence of a Newton-type method with the stability of gradient-descent-like behavior through a damping parameter. When an update successfully reduces the error, the damping parameter is decreased, allowing the algorithm to take more aggressive Newton-like steps. If an update increases the error, the update is rejected and the damping parameter is increased, making subsequent steps more conservative. This process is repeated until the predicted landing location is within the specified tolerance of the target or the maximum number of iterations is reached.

The solver also incorporates physical constraints into the optimization. Bounds can be placed on the ball’s launch speed, launch angles, and angular velocity, while the speed or spin can optionally be held constant. After each proposed update, these parameters are constrained to their allowable ranges before the next RK4 simulation is performed. This prevents the optimizer from finding mathematically valid but physically unattainable solutions (such as going through the hoop).

2.4 Trajectory Surfaces and Surrogate Modeling

Although the Levenberg–Marquardt solver can determine the required launch angles for a given robot state in real time, repeatedly solving the nonlinear projectile model can be computationally expensive and impractical for live, in-game calculations. To reduce this computational cost, we precompute a set of trajectory surfaces that describe the relationship between the robot’s motion, its distance from the target, and the required launch angles. These surfaces act as a surrogate for the full inverse solver, allowing the robot to estimate appropriate launch parameters through interpolation rather than performing a complete optimization at every shot. Actually, later we fit a polynomial over these surfaces for even faster approximation.

For each fixed shooter speed and spin, the model considers three independent inputs describing the robot’s state relative to the target: radial distance, radial velocity, and tangential velocity. Radial distance is the distance from the robot to the target measured along the direction pointing toward the target. Radial velocity is the component of the robot’s velocity in this same direction, while tangential velocity is the component perpendicular to it. Thus, the three-dimensional input space is

$$(r, v_f, v_l),$$

where r is radial distance, v_f is forward (radial) velocity, and v_l is lateral (tangential) velocity. The output of the inverse model is the pair of launch angles required for the shot,

$$(\theta, \phi).$$

For each point in this input space, the projectile solver is initialized with the fixed shooter speed and spin and with the robot’s radial and tangential velocity added to the launch velocity. The Levenberg–Marquardt solver then varies θ and ϕ while holding the shooter speed and spin fixed. The forward projectile simulation is repeatedly evaluated until the resulting trajectory reaches the target location. The resulting values of θ and ϕ are stored at the corresponding location in the three-dimensional input space.

The surfaces are generated over a discrete grid of radial distances and robot velocities. In the implementation, radial distance is sampled from 0 to 6.5 m (shortest to longest possible shot distance on the field), while both radial and tangential velocities are sampled from -5.5 to 5.5 m/s (range of possible robot velocities). Each valid grid point therefore represents one complete inverse ballistic solution.

Two separate three-dimensional data sets are produced: one mapping

$$(r, v_f, v_l) \rightarrow \theta$$

and another mapping

$$(r, v_f, v_l) \rightarrow \phi.$$

Together, these surfaces form a lookup representation of the inverse projectile model. During operation, the robot can use its measured distance and velocity to locate the corresponding region of the precomputed surfaces and estimate the required launch angles. This replaces the need to run the full Levenberg–Marquardt optimization for every shot, substantially reducing the computational work required during a match.

Because the surfaces are generated by repeatedly evaluating the physics-based projectile model, they retain the effects of gravity, aerodynamic drag, Magnus force, and rotational drag torque incorporated into the underlying simulation. The surface generation therefore does not replace the physics model with an empirical fit or machine-learning model; instead, it precomputes solutions to the physics-based inverse problem over the range of robot states expected during competition.

The surfaces are generated separately for different shooter operating points by converting the flywheel rotational speed into the corresponding ball launch speed and backspin. This allows a family of surfaces to be generated for different shooter speeds, providing a collection of precomputed inverse models that can be selected according to the desired shooter operating point.

The resulting trajectory surfaces are specifically parameterized for the 2026 FRC game and the physical characteristics and operating constraints of Team 900 The Zebracorns’ robot. They therefore provide a computationally efficient representation of the ballistics model over the region of the robot’s state space that is relevant during competition.

The following figures show both θ and ϕ surfaces for a projectile with a default launch speed of 50 RPS (rotations per second) of the shooter flywheel. The RPS values were converted into meters per second using the gear ratios of our robot at the time of the surface modeling. **IMPORTANT:** the following figures note θ as the **azimuthal angle** and ϕ as the **elevation angle**. This is the **opposite** convention as the text in this paper.

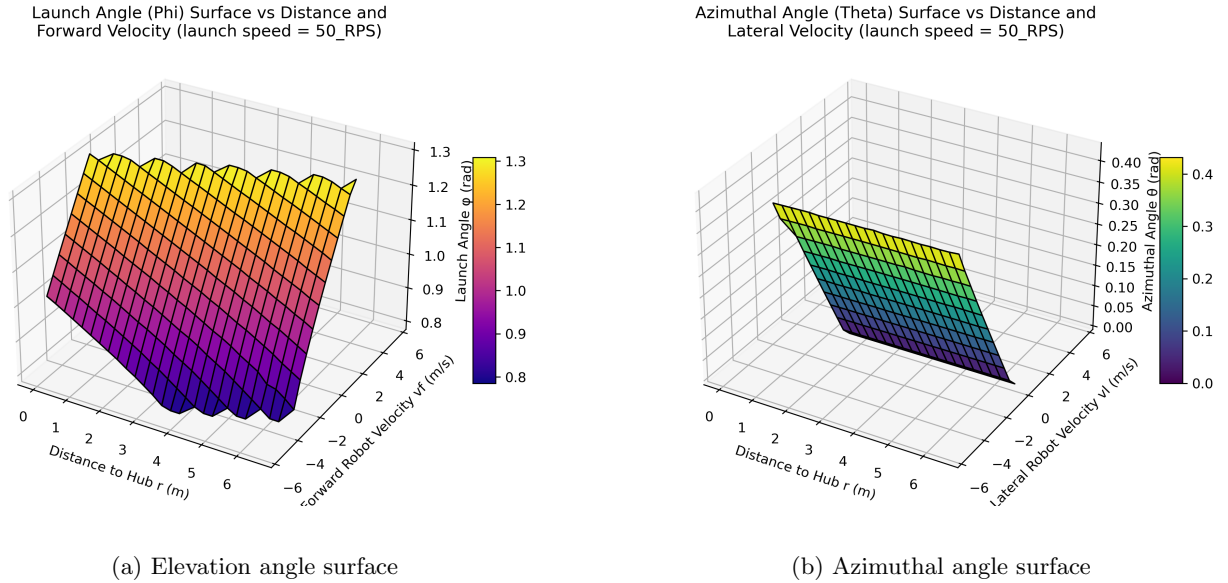


Figure 5: Surfaces (50 RPS) relating ϕ and θ to r , v_f , and v_l .

2.5 Polynomial Surface Fitting

The trajectory surfaces generated by the inverse solver provide a discrete set of solutions rather than a continuous mathematical function. While these surfaces can be used as lookup tables, storing and searching a three-dimensional grid is not ideal for real-time execution on the robot. To obtain a compact and computationally efficient representation, the simulated surfaces are fitted with multivariable polynomial models.

Before fitting, invalid simulation results are removed from the dataset. Points for which the Levenberg–Marquardt solver failed to find an accurate trajectory are represented as NaN values and are discarded. Solutions outside the physically valid launch-angle ranges are also removed. The remaining data is then filtered for outliers using a Tukey interquartile-range (IQR) fence independently for each radial-distance slice. This prevents isolated unsuccessful or anomalous solutions from significantly affecting the fitted model.

A polynomial regression model is then fit to the remaining trajectory data. For the elevation angle, the model uses all three measured state variables,

$$\theta = f(r, v_f, v_l),$$

because radial distance, radial velocity, and tangential velocity all affect the required elevation angle. The azimuthal angle is fit using only radial distance and tangential velocity,

$$\phi = g(r, v_l),$$

as the simulated data showed that ϕ is essentially independent of radial velocity. This reduces the dimensionality of the azimuthal model without significantly changing its predicted value.

The polynomial models are generated using fourth-degree polynomial features. Rather than fitting only individual powers of each variable, the model includes combinations of the input variables. In general, the fitted model has the form

$$f(x_1, x_2, \dots) = c_0 + \sum_i c_i \prod_j x_j^{p_{ij}},$$

where c_0 is the intercept, c_i are fitted coefficients, and p_{ij} are the integer powers associated with each polynomial term. For example, the elevation model can contain terms involving powers and combinations of r , v_f , and v_l , allowing it to approximate the nonlinear shape of the simulated trajectory surface.

The polynomial coefficients are obtained through linear regression after transforming the original inputs into their polynomial features. The resulting model therefore provides a continuous approximation to the discrete solutions generated by the physics simulation. The coefficient and power arrays are exported to JSON files, allowing the polynomial to be evaluated directly by the robot code without requiring the original Python simulation or machine-learning libraries.

At runtime, evaluating the model requires only the measured radial distance and robot velocity components. The polynomial evaluator computes each term,

$$c_i r^{p_{i,r}} v_f^{p_{i,v_f}} v_l^{p_{i,v_l}},$$

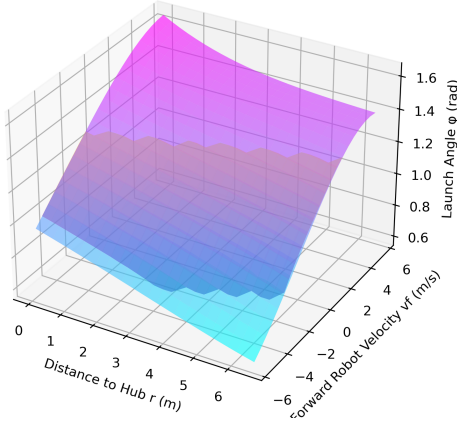
and sums the resulting values with the intercept. The predicted θ and ϕ can be calculated with simple arithmetic operations, rather than repeatedly running the RK4 integration and Levenberg–Marquardt optimization.

The quality of each polynomial approximation is evaluated using the coefficient of determination, R^2 , between the simulated surface values and the polynomial predictions. An R^2 value close to 1 indicates that the polynomial reproduces the simulated solutions closely over the fitted data set. This validates the compact polynomial model as a near-perfect approximation of the original physics-based inverse model.

The polynomial is not trained from experimental shot data and is not a machine-learning model in the sense of learning an unknown physical relationship. Instead, it is a mathematical approximation of solutions generated by the physics-based inverse solver. This allows the computationally expensive physics simulation to be performed offline while retaining a fast analytical approximation that can be evaluated during a match.

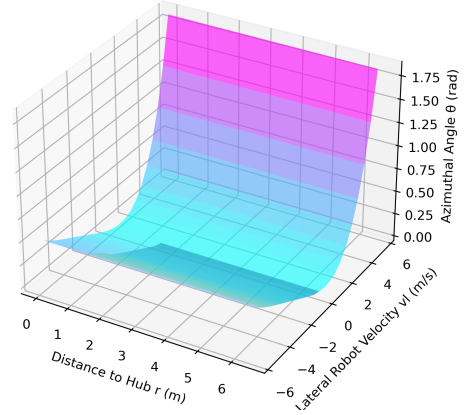
The following figures show both θ and ϕ surfaces with each respective polynomial fit overlaid for a projectile with a default launch speed of 50 RPS (rotations per second) of the shooter flywheel. The RPS values were converted into meters per second using the gear ratios of our robot at the time of the surface modeling. **IMPORTANT:** the following figures note θ as the **azimuthal angle** and ϕ as the **elevation angle**. This is the **opposite** convention as the text in this paper.

Phi: Data (plasma) vs Polynomial Fit (cool) (launch speed = 50_RPS)



(a) Elevation angle surface with fit

Theta: Data (viridis) vs Polynomial Fit (cool) (launch speed = 50_RPS)



(b) Azimuthal angle surface with fit

Figure 6: Surfaces and polynomial fits (50 RPS) relating ϕ and θ to r , v_f , and v_l .

The complete project pipeline can be viewed as the following sequence:

Physics Model $\rightarrow$ RK4 Approximation $\rightarrow$ Levenberg–Marquardt Inverse Solver $\rightarrow$ Trajectory Surfaces $\rightarrow$
Valid-Shot Selection $\rightarrow$ Polynomial Models.

3 Results and Discussion

3.1 Model Results

The completed ballistics mapper provides a physics-based method for determining the launch conditions required to place a ball at the target location. The forward model accounts for gravity, aerodynamic drag, the Magnus force, and rotational drag torque, allowing the simulated trajectory to capture effects that would be difficult to represent using a simple projectile-motion equation. The Levenberg–Marquardt inverse solver then uses this forward model to determine the launch conditions required to reach the target.

Rather than executing this computationally expensive inverse solver repeatedly during a match, the solutions are precomputed over the range of robot states expected during competition. The resulting trajectory surfaces describe how the required elevation angle θ and azimuthal angle ϕ vary with radial distance, radial velocity, and tangential velocity for a fixed shooter operating point. These surfaces are subsequently approximated using fourth-degree multivariable polynomial models, allowing the robot to evaluate the required launch angles using only a small number of arithmetic operations.

The polynomial representation substantially reduces the computational requirements of the ballistic calculation. The full physics simulation and inverse optimization can be performed offline, while the resulting polynomial coefficients can be evaluated directly by the robot’s control software. This makes it possible to retain the behavior of the physics-based model without requiring the computational cost of repeatedly running the projectile simulation and Levenberg–Marquardt solver during a match.

The shooter-speed selection process further extends this approach by evaluating the available shooter operating points for each combination of robot distance and velocity. Instead of assuming that there is a single shooter speed that works for every state, the resulting valid-shot table identifies every shooter speed for which the corresponding elevation and azimuthal angles remain within the mechanical limits of the robot. This allows the higher-level robot control system to select among multiple physically achievable shooting solutions.

3.2 Numerical Validation

To verify that the precomputed trajectories solved using Levenberg-Marquardt remain consistent with the underlying numerical model, a separate numerical verification procedure samples previously generated valid trajectories and reruns them through the projectile simulator. For each randomly selected state, the stored radial distance, robot velocity, shooter speed, and spin are used to reconstruct the launch conditions. The resulting projectile trajectory is then simulated using the fourth-order Runge-Kutta (RK4) model. The predicted landing position is compared with the target position, and the resulting position errors are reported. A fixed random seed is used so that the verification set is reproducible.

3.3 Experimental Validation

The radial-velocity component of the model was successfully validated on the physical robot. In particular, the model’s prediction of the required elevation angle θ as a function of radial robot velocity produced immediate successful results during testing. This demonstrated that the model was able to account for the effect of the robot’s motion toward or away from the Hub and translate that motion into an appropriate change in launch elevation.

This result was significant because the elevation model was not simply fitted to observed shot data. The required launch conditions were generated from the physics-based projectile model and inverse solver, then reduced to a computationally efficient polynomial representation. The successful radial-velocity tests therefore provided practical evidence that this portion of the physics-based model was sufficiently accurate for use on the robot.

The tangential-velocity component was not experimentally validated to the same extent. The model predicts that tangential motion relative to the Hub changes the required azimuthal launch angle ϕ . However, mechanical and software difficulties with the turret prevented reliable testing of the turret’s azimuthal response while the robot was moving with controlled tangential velocity.

Consequently, the radial-velocity/elevation portion of the moving-shot model was demonstrated successfully on the physical robot, while the tangential-velocity/azimuthal portion remains a physics-based prediction that requires further experimental validation. This distinction is important when evaluating the overall model: the lack of testing for the azimuthal component was a limitation of the robot implementation and testing process rather than an indication that the underlying relationship was unsuccessful.

3.4 Limitations

The most significant experimental limitation of the current system was the inability to reliably test the tangential-velocity compensation. The turret’s mechanical and software limitations prevented controlled experiments in which the robot could move laterally relative to the Hub while simultaneously commanding and measuring the predicted azimuthal angle ϕ .

In contrast, the radial-velocity and elevation component was successfully demonstrated on the physical robot. The immediate success of this component provides experimental support for the model’s treatment of radial robot velocity and its effect on the required launch elevation.

The accuracy of the remaining model is also dependent on the physical parameters used to describe the Fuel and its interaction with the surrounding air. Aerodynamic coefficients, ball-to-ball variation, shooter behavior, and other mechanical effects can introduce differences between the simulated and physical trajectories. The model therefore represents an approximation of the physical system rather than an exact description of every factor affecting the shot.

Another limitation is that the polynomial models are only intended to operate within the state space represented by the precomputed trajectory surfaces. States outside this region may produce inaccurate or undefined predictions. The offline preprocessing approach therefore requires the relevant range of robot positions and velocities to be sufficiently covered before competition use.

3.5 Significance and Applications

The primary significance of this approach is that it provides a middle ground between a purely analytical projectile model and a purely empirical shooting model. An empirical approach can be effective but requires

collecting successful-shot data over a large portion of the robot’s operating range. A fully physics-based inverse solver provides a more general model but can be too computationally expensive to execute repeatedly in real time. The trajectory-surface and polynomial approach combines the two advantages: the underlying solutions are generated from physics, while the final representation is compact enough for real-time use.

The same approach can also be applied beyond the specific FRC 2026 game. Any robotic system that launches a projectile toward a known target can potentially use a similar pipeline. A forward physical model can be developed for the projectile, an inverse solver can determine the launch conditions required to reach a target, and the resulting solutions can be precomputed and approximated for fast runtime evaluation.

The method is particularly useful when the robot or launcher is moving. In a stationary system, the target direction alone largely determines the required launch direction. When the launcher is moving, however, the velocity of the launcher contributes to the projectile’s inertial velocity. Decomposing this velocity into radial and tangential components provides a natural way to separate the effects on elevation and azimuth. This makes the approach applicable to other mobile robotic systems where accurate projectile placement must occur without first stopping the robot.

The polynomial representation also provides a practical way to deploy a relatively complicated physical model on hardware with limited computational resources. Once the coefficients have been generated offline, the robot does not need to store the complete simulation or execute an iterative nonlinear optimizer. Instead, it evaluates the polynomial models using the currently measured robot state and uses the resulting launch angles as inputs to the shooter and turret controllers.

3.6 Future Refinements

The most immediate improvement to the system would be completing experimental validation of the tangential-velocity and azimuthal-angle relationship. A mechanically reliable turret would allow controlled experiments in which the robot moves with known tangential velocities while shooting at the hoop. The measured azimuthal error could then be compared directly against the model’s predicted ϕ values.

Once this behavior is experimentally validated, the model could be refined using measured shot data to identify systematic differences between the theoretical model and the physical robot. For example, experimental data could be used to refine aerodynamic coefficients, characterize mechanical offsets, or compensate for systematic turret positioning errors. This would preserve the physics-based structure of the model while improving its agreement with the actual robot.

Another potential refinement would be increasing the resolution of the precomputed trajectory surfaces in regions where the launch angles change rapidly. The current surfaces discretize the robot’s state space, so increasing the sampling density could reduce interpolation or approximation error. Conversely, regions in which the solution changes slowly could potentially use a lower sampling density, reducing the size of the generated data.

The polynomial approximation could also be evaluated using different polynomial degrees or alternative approximation methods. A fourth-degree polynomial provides a compact representation, but increasing the degree could improve accuracy in some regions while increasing the number of coefficients and the computational cost of evaluation. The appropriate degree should therefore be selected based on the tradeoff between accuracy, memory usage, and runtime performance on the robot.

Finally, the shooter-speed selection system could be extended to include a more explicit optimization criterion. Currently, the valid-shot table identifies every shooter speed capable of producing a mechanically achievable trajectory. A higher-level controller could use this information to select the best solution according to objectives such as minimizing shooter speed, maximizing available trajectory margin, reducing spin requirements, or maintaining a preferred operating point for the shooter.

Overall, the current system establishes a framework in which a detailed physics simulation can be converted into a practical real-time shooting model. Completing the experimental validation of the moving-shot azimuthal behavior would be the most important next step toward demonstrating the full capability of the system on the physical robot and is left as an exercise for the reader.

4 Final Remarks

The 2026 FRC season for Team 900 was nothing short of extraordinary. We had a lot of high-risk, high-reward decisions made, and unfortunately, many of those decisions did not pay off. Due to various software and mechanical shortcomings, we were unable to reach the state-level competition. This was certainly a humbling experience, and we learned a lot as a team. While there is no solid, continuous match footage to demonstrate the ballistic mapper, clips of it working well can be found during certain portions of certain matches. The source code for this project can be found at: <https://github.com/anshmenghani/BallisticsMapper>.